AIKernel Phase-1 Paper 04: Trajectory Governance Model

A Formal Specification for Deterministic Execution Control over Stochastic AI Agents

Takuya Sogawa

2026-05-26

AIKernel Phase-1 Paper 04: Trajectory Governance Model

A Formal Specification for Deterministic Execution Control over Stochastic AI Agents

Author: Takuya Sogawa
ORCID: https://orcid.org/0009-0009-7499-2595
Version: v0.2.0
Publication date: 2026-05-26
DOI: https://doi.org/10.5281/zenodo.20309510

Canonical language: English
Companion translation: Japanese
Series position: AIKernel / AIOS Phase-1 Specification Series, Part II-4: Governance Layer
Target: AIKernel.NET Governance / Trajectory
Proposed namespace: `AIKernel.Abstractions.Governance.Trajectory`
Stability: Experimental / Non-normative
Based on: Trajectory Governance Model v0.1.1, DOI: https://doi.org/10.5281/zenodo.20223205

---

1. Abstract

This paper defines the Trajectory Governance Model in AIKernel / AIOS Phase-1. Trajectory Governance is a kernel-level governance model that observes LLM / SLM outputs, candidate actions, semantic state transitions, goal alignment, risk, uncertainty, and repetition after inference has begun, and determines whether a proposed transition may be accepted as an executable system transition.

This model does not attempt to make probabilistic generation itself deterministic. Instead, it evaluates stochastically proposed state transitions through a deterministic Policy Decision Point and maps them into controlled governance decisions: `Permit`, `Deny`, `AskConfirmation`, `Clarify`, or `Abort`. In this way, AIKernel does not treat stochastic inference as directly executable; it improves the safety, reproducibility, and auditability of execution transitions through observable semantic trajectories.

> **Relationship to the prior DOI version**
> This paper is the v0.2.0 series edition of the Trajectory Governance Model, reorganized as Paper 04 of the AIKernel / AIOS Phase-1 Paper Series.
> The prior standalone version is archived at DOI: `10.5281/zenodo.20223205`.
> This v0.2.0 edition preserves the core Trajectory Governance theory while reorganizing its scope, terminology, dependencies, and structure for the Phase-1 paper series.

2. Introduction

The rapid development of artificial intelligence, especially large language models (LLMs), marks a major shift in the design of computational systems. AI agent systems built around LLMs provide advanced natural-language intent interpretation and dynamic problem-solving capabilities. At the same time, they

exhibit non-deterministic behavior because they operate through stochastic inference processes and unstructured semantic data.

This non-determinism creates major challenges for enterprise adoption, especially with respect to reliability, reproducibility, auditability, and runtime safety. Traditional safety approaches often rely on input filters, final-output checks, static prompt engineering, or external guardrails. However, these approaches do not necessarily provide a model for continuously observing and governing the inference process itself as a sequence of state transitions.

The AIKernel project is designed as a framework that introduces auditable control boundaries around AI execution environments. This paper formalizes Trajectory Governance as a core model of the AIKernel Governance Layer. It provides a deterministic, fail-closed control boundary for issues such as context contamination, goal drift, inference divergence, and unsafe candidate actions.

The LLM is treated as a stochastic proposal generator, while AIKernel acts as the deterministic governance boundary that decides whether proposed transitions may affect executable system state.

3. Scope and Boundary

Trajectory Governance governs semantic state transitions and candidate actions after inference has begun in the AIKernel / AIOS execution pipeline.

3.1 Inbound Boundary

The requests handled by this model are assumed to have already passed the pre-inference checks defined in Paper 03: Pre-Inference Admissibility Governance. In other words, the input is assumed to have passed at least the following entry controls:

- Prompt Injection / Override Gate
- Capability Admission Gate
- Critical Operation Gate
- Computational Complexity Gate

In particular, if the task itself exceeds the `ModelExecutionBudget`, i.e., the model's reliable operating envelope, it is handled at the Paper 03 stage as `DelegateToSolver`, `Decompose`, `Clarify`, or `Deny`. Paper 04 addresses state transitions and candidate actions that occur inside an already accepted inference transaction.

3.2 Outbound Boundary

Trajectory Governance evaluates candidate actions, generated intermediate states, tool calls, context hydration candidates, and final execution candidates proposed by an LLM / SLM. These are treated as untrusted until they are evaluated by the governance layer.

This model maps candidate states into the following controlled governance decisions:

- `Permit`
- `Deny`
- `AskConfirmation`
- `Clarify`
- `Abort`

3.3 Non-Goals

Trajectory Governance does not aim to:

- prove the factual correctness of LLM outputs;
- prove the semantic completeness of embedding models;
- provide complete protection against all adversarial inputs;
- make the internal inference process of an LLM fully deterministic;
- replace the Pre-Inference Governance model defined in Paper 03;

- fully define the canonical ReplayLog schema or Execution Model defined in Paper 05.

4. Terminology

The following terms define the core concepts used in the AIKernel architecture.

- **PDP / PEP / PIP**: Standard components in access-control architecture. A PDP (Policy Decision Point) evaluates policy decisions. A PEP (Policy Enforcement Point) enforces those decisions at execution boundaries. A PIP (Policy Information Point) provides attributes, contextual information, and risk signals needed for policy evaluation.
- **Trajectory**: A sequence of semantic states generated during the inference process of an LLM.
- **Semantic Ellipsoid**: A geometric model that represents queries, documents, states, or candidate actions not as single point vectors, but as semantic regions with uncertainty.
- **RootGoal**: An immutable goal state representing the user's fundamental intent.
- **WorkingGoal**: The current task interpretation managed by context at a given inference step.
- **Convergence Score**: An operational scalar estimate indicating whether the inference trajectory is progressing stably toward the intended goal.
- **Anomaly Score**: A scalar estimate used to detect local anomalies in a trajectory, such as sharp drift, entropy spikes, or policy-violation tendencies.
- **ModelExecutionBudget**: The limit or execution budget, defined in Paper 03, that determines which levels of computational complexity should be handled by an LLM.
- **ReplayLog**: An immutable record of metrics, thresholds, and decisions used for deterministic replay and auditability.
- **SGP Pipeline**: The AIKernel inference-separation model that separates Structure, Generation, and Polish phases, ensuring that logical structure and surface expression are handled as distinct stages.

5. Safety Boundary

This model does not claim that a probabilistic generator such as an LLM can be made fully deterministic. It also does not claim that semantic convergence unconditionally guarantees factual correctness.

Instead, AIKernel defines a deterministic safety boundary around the stochastic inference process. Model outputs are treated as untrusted until the governance layer evaluates semantic relevance, goal alignment, operational risk, repetition, uncertainty, and replayability.

In this sense, Trajectory Governance does not replace internal model safety. Rather, it functions as a kernel-level control plane that determines whether a model-proposed state transition may be accepted as an executable system transition.

6. Problem Statement

Current autonomous AI systems face the following theoretical and operational challenges.

1. **Lack of semantic boundaries**: Due to attention mechanisms and long context chains, AI systems are vulnerable to context contamination and injection attacks.
2. **Opacity of state transitions**: There is a lack of computational models for tracking and controlling repetition, uncertainty growth, and inference drift.
3. **Goal Drift**: Inference may gradually diverge from the user's original intent, with no structural mechanism to prevent such drift.
4. **Risk of candidate actions**: If candidate actions or tool calls generated by an LLM are treated directly as executable state transitions, they may cause irreversible side effects.
5. **Non-deterministic safety evaluation**: Policies are often written in natural language, making system behavior difficult to predict or reproduce.

This paper defines an implementable governance model that represents AI inference as a continuous trajectory and evaluates system stability and safety through deterministic governance metrics.

7. Mathematical Formalization of Trajectory Governance

The convergence scores and mathematical definitions introduced in this model do not prove correctness or safety themselves. They are operational policy scores used to determine whether an observed inference trajectory is sufficiently stable, aligned, and low-risk under configured governance parameters.

7.1 Formalizing LLM Inference and the Four-Stage Governance Intervention Model

AIKernel formalizes the LLM inference process as a stochastic transition function dependent on an internal state K_t and model parameters θ.

$$x_{t+1} \sim p(x \mid x_t, \theta, K_t)$$

AIKernel applies the following four-stage governance intervention model to this stochastic transition process.

1. **Propose**: The LLM / SLM stochastically proposes the next action or state x_{t+1}.
2. **Evaluate**: The governance layer computes risk, convergence, and goal alignment for the proposed state.
3. **Decide**: The PDP classifies the proposed state transition into one of the controlled governance decisions: `Permit`, `Deny`, `AskConfirmation`, `Clarify`, or `Abort`.
4. **Transition**: Only if permitted, the transition is committed as a system state.

7.2 Geometric Properties and Implementation Approximation of Semantic Ellipsoids

Each state or candidate on the trajectory is represented not as a single point vector, but as a semantic region that includes uncertainty. Because computing a full covariance matrix Σ is expensive in high-dimensional spaces, a diagonal covariance approximation may be used in implementation.

A semantic state s_t is represented as a pair consisting of a center vector μ_t and a diagonal covariance approximation Σ_t.

$$s_t = (\mu_t, \Sigma_t)$$

Given semantic samples or candidate vectors generated by an LLM, $x_1, x_2, \dots, x_N$, the center and variance are estimated as follows.

$$\mu_t = \frac{1}{N} \sum_{i=1}^{N} x_i$$

$$\Sigma_t = \mathrm{diag}(\sigma_{t,1}^2, \sigma_{t,2}^2, \dots, \sigma_{t,n}^2)$$

$$\sigma_{t,j}^2 = \frac{1}{N} \sum_{i=1}^{N} (x_{i,j} - \mu_{t,j})^2$$

The approximate distance from an arbitrary semantic vector x to state s_t can be computed using a stabilizing constant $\epsilon > 0$.

$$d(x, s_t) = \sqrt{\sum_{j=1}^{n} \frac{(x_j - \mu_{t,j})^2}{\sigma_{t,j}^2 + \epsilon}}$$

The admissible semantic region is represented using a control coefficient $k > 0$.

$$\mathcal{E}_t(k) = \{x \in \mathcal{V} \mid d(x, s_t) \leq k\}$$

This model does not assume that an embedding space perfectly represents human meaning. A Semantic Ellipsoid is an implementation-oriented approximation for handling observable semantic variance.

7.3 Three Axes of Drift

Trajectory drift is defined along the following three axes.

1. **Gradient Δ_t**: The rate of change of the center vector between consecutive semantic states.
2. **Directional vector**: A cosine-based measure or directional smoothness metric indicating whether inference is progressing toward the goal.
3. **Normalized entropy $\tilde{H}_t$**: The degree of uncertainty derived from token probabilities, logits, or proxy metrics.

Normalized entropy is bounded into the $[0, 1]$ range by dividing by the maximum entropy $\log |V|$ based on vocabulary size $|V|$.

$$H_t = -\sum_{w \in V} p(w) \log p(w)$$

$$\tilde{H}_t = \frac{H_t}{\log |V|}$$

In black-box API mode or lightweight local-model mode, full logits or hidden states may not be observable. In such cases, JSON validity, response stability, candidate repetition, and embedding displacement may be used as proxy metrics.

7.4 Definition of Convergence Score C_t and Anomaly Score A_t

The observed metrics are integrated into two core governance scores. Each score is normalized and always bounded within $[0, 1]$.

Convergence Score C_t indicates whether the trajectory is progressing stably toward the goal.

$$\begin{aligned} C_t = \text{Clamp}\big(&\alpha \cdot \text{SemOverlap}_t + \beta \cdot \text{GoalAlign}_t \\ &- \gamma \cdot \text{SmoothDrift}_t - \delta \cdot \tilde{H}_t - \rho \cdot \text{Risk}(a_t) \\ &- \eta \cdot \text{Rep}_t - \zeta \cdot \text{GoalDrift}_t, 0, 1\big) \end{aligned}$$

Anomaly Score A_t is independent of the convergence score and detects local risk spikes, sharp drift, entropy explosions, or policy-violation tendencies.

$$\begin{aligned} A_t = \text{Clamp}\big(&\omega_1 \cdot \text{RawDriftSpike}_t + \omega_2 \cdot \text{EntropySpike}_t \\ &+ \omega_3 \cdot \text{RiskSpike}_t, 0, 1\big) \end{aligned}$$

7.5 Consecutive Threshold Violation and Fail-Closed Conditions

AIKernel defines the Governance Cost Function / Safety Index as follows.

$$V_t = 1 - C_t$$

AIKernel does not claim that V_t is a strict Lyapunov function or a Barrier Certificate in the sense used in dynamical systems. It is an operational evaluation function defined over observable governance metrics.

Therefore, a decreasing trend in V_t may be treated as an operational sign of stabilization, but it does not prove global mathematical stability. The Mahalanobis-style distance and Semantic Ellipsoid approximation used in this paper are observability primitives for bounded deterministic governance; they are not presented as control-theoretic stability proofs.

When a trajectory deviates beyond the safety boundary, the system immediately aborts execution according to the following logical condition.

$$\text{Abort} = (C_t < \tau_C) \vee (A_t > \tau_A) \vee (\text{Risk}(a_t) \geq \tau_R) \vee (\text{Rep}_t > \tau_P) \vee (\text{GoalDrift}_t > \tau_{root})$$

To avoid false positives caused by temporary fluctuations, the system may also fall back to a fail-closed state if $C_t < \tau_C$ occurs at least m times within a time window of W steps.

8. Goal Governance and Context Sovereignty

To prevent goal drift, AIKernel governs goal states under Context Sovereignty and defines an auditable control boundary through the following hierarchy.

- **Computational alignment of RootGoal and WorkingGoal**:
  - **RootGoal (g_{root})** represents the user's fundamental intent and is defined as immutable.
  - **WorkingGoal ($g_{work,t}$)** represents the step-specific interpretation of the current task. In implementation, it is constrained by projection into the admissible semantic region of g_{root} or by threshold-based validation.
  - Goal drift is defined as drift $= 1 - \cos(g_{root}, g_{work,t})$.
  - A request to update the WorkingGoal is permitted only if drift remains below the configured threshold. Otherwise, it is rejected.

Under this rule, an LLM or SLM may propose an interpretation at an inference step, but it has no authority to modify the RootGoal itself. Updates to the WorkingGoal are always constrained by semantic distance from the RootGoal.

9. Security Model and Risk Assessment

9.1 Initial Risk Evaluation and Calibration

AIKernel adopts the standard PDP / PEP / PIP access-control architecture for inference execution decisions. Initial risk evaluation is category-based and deterministic. Destructive operations such as file deletion, or high-entropy inference results, are assigned higher base risk scores.

Statistical calibration based on ReplayLog data may later be used to adjust weights, but it must not replace hard policy constraints, fail-closed rules, or capability boundaries.

9.2 Candidate Generation and Filtering

At inference step t, the LLM or SLM generates a set of candidates that may include the next action, a tool-execution command, or the next-stage reasoning context.

$$\text{Candidates}_t = \{a_1, a_2, \dots, a_n\}$$

These candidates are not immediately permitted to execute or hydrate into context. They are temporarily held in an evaluation space and are treated as executable candidates only if they pass static validation, risk evaluation, trajectory evaluation, and goal-alignment evaluation.

Candidates that violate static validation rules are rejected before scoring.

$$\text{Candidates}_t^{filtered} = \{a_i \in \text{Candidates}_t \mid \text{IsValidSyntax}(a_i) \wedge \text{IsAuthorizedCapability}(a_i)\}$$

9.3 Decision Rule and Candidate Ranking

When multiple candidate actions are proposed by the LLM / SLM, the PDP ranks candidates and makes a final decision based on the following rule set.

In this paper, **Candidates** refers to the set of LLM / SLM-proposed actions subject to governance evaluation.

Any action whose risk is greater than or equal to τ_R is excluded from the executable candidate set. The ranking score for each candidate action is defined using action-specific semantic alignment and risk while sharing the current trajectory state.

$$\begin{aligned}\text{Score}(a) =& \alpha \cdot \text{SemOverlap}(q, a) + \beta \cdot \text{GoalAlign}_t \\ & - \gamma \cdot \text{SmoothDrift}_t - \delta \cdot \tilde{H}_t - \rho \cdot \text{Risk}(a) \\ & - \eta \cdot \text{Rep}_t - \zeta \cdot \text{GoalDrift}_t\end{aligned}$$

The safe candidate set is defined as:

$$A_{safe} = \{a \in \text{Candidates} \mid \text{Risk}(a) < \tau_R\}$$

The selected candidate is:

$$a^* = \arg \max_{a \in A_{safe}} \text{Score}(a)$$

If $A_{safe} = \emptyset$, arg max is not evaluated, and the system returns `Deny` or `Clarify`.

For the selected a^*, the PDP makes the final governance decision using the following priority order. Here, $\tau_{safe} < \tau_R$. Risk above τ_{safe} but below τ_R is treated as requiring confirmation rather than immediate denial.

$$\begin{aligned}&\text{if Abort} = \text{true} && \Rightarrow \text{Return}(\text{Abort}) \\ &\text{else if SemOverlap}_t < \tau_{\text{sem}} && \Rightarrow \text{Return}(\text{Clarify}) \\ &\text{else if Risk}(a^*) > \tau_{\text{safe}} && \Rightarrow \text{Return}(\text{AskConfirmation}) \\ &\text{else if } C_t \geq \tau_{\text{exec}} && \Rightarrow \text{Return}(\text{Permit}) \\ &\text{else} && \Rightarrow \text{Return}(\text{Deny})\end{aligned}$$

Decision	Meaning
`Permit`	Execution is permitted.
`AskConfirmation`	Human confirmation is required due to medium risk.
`Clarify`	Clarification is required due to semantic ambiguity.
`Deny`	Execution is rejected by policy.
`Abort`	Execution is stopped under fail-closed conditions.

10. Theoretical Guarantees and Proof Sketches

Based on the mathematical model and governance architecture described above, AIKernel provides the following theoretical guarantees.

10.1 Assumptions

- **A1**. All governance metrics, scores, and penalties are normalized and bounded within $[0, 1]$.
- **A2**. Governance weights and decision thresholds are fixed within a single decision step.
- **A3**. Operational risk $R(a)$ is evaluated for every action before any side-effectful execution.
- **A4**. The governance decision function I_{PDP} is fully deterministic.
- **A5**. An LLM or SLM may propose candidates, but it has no authority to execute them directly.
- **A6**. Tools and system calls are executed only when the PDP decision is `Permit`.

10.2 Theorem 1. Normalization Property of Scoring Function

The convergence score C_t and anomaly score A_t always satisfy $0 \leq C_t \leq 1$ and $0 \leq A_t \leq 1$ as final governance outputs.

Proof Sketch: By A1, all input parameters are bounded within $[0, 1]$, so their linear combination is finite. However, because the convergence score includes negative penalty terms such as risk and repetition, the raw linear combination is not guaranteed to remain within $[0, 1]$. Applying $\text{Clamp}(x, 0, 1)$ as the final step guarantees that the final C_t and A_t passed to the system are always normalized within $[0, 1]$.

10.3 Theorem 2. Fail-Closed Safety

If governance evaluation determines $\text{Abort} = \text{true}$, no proposed action a is executed.

$$\text{Abort} = true \Rightarrow \neg\text{Exec}(a)$$

Proof Sketch: By A5 and A6, execution authority is isolated in the PEP. The PEP implementation is architecturally required to block any side-effectful API path unless the PDP decision is `Permit`. Therefore, if `Abort` is selected, no path exists for a stochastic output to reach the physical environment.

10.4 Theorem 3. Risk Threshold Exclusion

An action whose computed risk is greater than or equal to the configured threshold is excluded from the safe execution candidate set A_{safe}.

$$R(a) \geq \tau_R \Rightarrow a \notin A_{safe}$$

Proof Sketch: By A3, $R(a)$ is computed for every candidate action before execution. During evaluation, A_{safe} is constructed using the predicate $R(a) < \tau_R$. Therefore, any action that fails this condition is deterministically excluded from the final arg max selection.

10.5 Theorem 4. Root Goal Sovereignty Invariant

As long as g_{root} is immutable and WorkingGoal updates are enforced by the PEP according to the drift threshold, updates to $g_{work,t}$ remain within the defined admissible semantic region.

Proof Sketch: For any requested WorkingGoal update at time t, the system evaluates:

$$\text{drift} = 1 - \cos(g_{root}, g_{work,t+1})$$

The state transition is permitted only if $\text{drift} \leq \tau_{drift}$. Otherwise, the update is rejected. Because the system evaluates distance from the immutable g_{root} on every update and the PEP blocks transitions outside the admissible semantic region, $g_{work,t}$ remains constrained within the admissible region defined around g_{root}.

10.6 Theorem 5. Deterministic Governance Replay

If implementation version, numerical precision, embedding model, policy, thresholds, weights, and input records are fixed, the governance decision is reproducible.

Proof Sketch: By A2 and A4, the governance decision function is deterministic. The ReplayLog records runtime metadata immutably, and stochastic variation or external factors are controlled through recorded replay inputs. As long as the specified environment conditions and numerical precision match, the same input set is evaluated as a composition of deterministic functions, producing the same decision.

10.7 What These Guarantees Do Not Prove

The AIKernel Trajectory Governance Model provides robust governance, but it does not mathematically prove the following:

- **Factual correctness of LLM outputs**: Whether generated text corresponds to real-world facts.
- **Semantic completeness of embeddings**: Whether vector embeddings capture human-intended meaning without error.
- **Complete robustness against adversarial inputs**: Whether all unknown prompt-injection attempts can be prevented.
- **Global mathematical stability**: Whether $V_t = 1 - C_t$ functions as a strict Lyapunov function or Barrier Certificate.

What AIKernel guarantees is the determinism of execution-transition authorization conditions: it evaluates and controls whether a proposed action complies with defined safety boundaries and governance parameters.

11. System Architecture and Formal Specification

For Trajectory Governance to operate in a real execution environment, the following components are specified at the architectural level.

- **Context Isolation and Material Quarantine**: Information categories are strictly separated, and external data is integrated into inference context only after passing through a quarantine process and structural normalization.
- **SGP Pipeline**: Structure, Generation, and Polish phases are separated so that logical structure and surface expression are handled in distinct stages.
- **PDP / PEP Boundary**: The LLM may propose candidates, but execution authority is isolated in the PEP.
- **ReplayLog Integration**: All governance decisions are recorded for later auditing, deterministic replay, and calibration.

12. Experimental Validation Plan

To validate the implementability of the theoretical model, the following phased validation plan is defined.

- **Phase 1: Connectivity**: Use a CPU-only lightweight local SLM or an OpenAI-compatible local inference server to verify connectivity and structured-output stability.
- **Phase 2: Candidate Generation**: Generate candidate actions from natural-language requests and test risk classification and candidate filtering.
- **Phase 3: Governance Validation**: Verify that fail-closed governance, confirmation routing, and Root-Goal drift detection behave as intended for proposed candidates.
- **Phase 4: Replay Calibration**: Use ReplayLog data to calibrate weights and thresholds.
- **Phase 5: Open-Weight Extension**: Use open-weight models such as Transformers or vLLM to capture hidden states, logits entropy, or top-k probability distributions, and evaluate their correlation with proxy metrics.

13. Limitations

This architecture provides a strong governance model, but it includes the following limitations.

- **Observability limitation**: In black-box API mode or lightweight local-model mode, the true hidden-state trajectory cannot be directly observed.
- **Embedding dependence**: Semantic drift and goal alignment depend on embedding quality.
- **Threshold sensitivity**: Weights and thresholds are initial values and should be calibrated using ReplayLog data.
- **Sampling cost and latency overhead**: Candidate sets and Semantic Ellipsoids may introduce additional generation, embedding, and scoring costs.
- **False sense of security**: A high convergence score does not guarantee factual correctness or operational safety.
- **Adversarial inputs**: Adversarial inputs may manipulate goal summaries, structured outputs, or candidate generation; therefore, capability restrictions and sandboxing must be used in combination.

14. Systematized AIKernel Theory: Semantic Context OS

The reconstructed AIKernel theory can be summarized through three pillars: semantic encapsulation, declarative governance, and resource abstraction with dynamic routing.

Trajectory Governance is a core element of declarative governance. In the broader AIKernel architecture, it functions as the Governance Layer that connects stochastic inference to verifiable execution transitions in the Semantic Context OS model.

15. Conclusion

The AIKernel Trajectory Governance Model is a formal governance model that introduces verifiability, auditability, and fail-closed execution control into the stochastic and non-deterministic nature of generative AI.

This model does not aim to make intelligence itself more powerful. Rather, it evaluates state transitions and candidate actions proposed by LLMs / SLMs under AIKernel's deterministic governance boundary, and decides whether they may be accepted as executable system transitions.

By integrating Context Isolation, Semantic Ellipsoids, RootGoal immutability, PDP / PEP architecture, and ReplayLog-based auditability, this model serves as a core Governance Layer component for constructing AIKernel as a Semantic Context OS.

References

1. Mahalanobis, Prasanta Chandra. "On the Generalized Distance in Statistics." Proceedings of the National Institute of Sciences of India, vol. 2, no. 1, 1936, pp. 49-55. Available at: https://insa.nic.in/writereaddata/UpLoadedFiles/PINSA/Vol02_1936_1_Art05.pdf.
2. National Institute of Standards and Technology. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, 2014. DOI: 10.6028/NIST.SP.800-162.
3. OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 22 January 2013. Available at: http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html.
4. Sikka, Varin, and Vishal Sikka. "Hallucination Stations: On Some Basic Limitations of Transformer-Based Language Models." arXiv:2507.07505, 2025. DOI: 10.48550/arXiv.2507.07505.
5. Sogawa, Takuya. "Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model." Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
6. Sogawa, Takuya. "AIKernel Phase-1 Paper 03: Pre-Inference Admissibility Governance." Zenodo, 2026. DOI: 10.5281/zenodo.20308639.
7. Sogawa, Takuya. "AIKernel Trajectory Governance Model: A Kernel-Level Framework for Convergent Decision Control over Stochastic Language Model Inference." Zenodo, 2026. DOI: 10.5281/zenodo.20223205.

Appendix A. Suggested Initial Parameter Values

The following values are suggested starting points for initial deployment and ReplayLog-based calibration. They are not universal constants; they should be adjusted based on operational logs.

Parameter	Initial Value	Meaning
α	0.30	Positive contribution of Semantic Overlap to the convergence score
β	0.20	Positive contribution of Goal Alignment
γ	0.20	Penalty coefficient for Smooth Drift
δ	0.10	Penalty coefficient for Normalized Entropy
ρ	0.15	Subtractive penalty coefficient for Operational Risk
η	0.05	Penalty coefficient for Repetition
ζ	0.10	Penalty coefficient for RootGoal drift
ω_1	0.40	Contribution of Raw Drift Spike to the anomaly score
ω_2	0.30	Contribution of Entropy Spike to the anomaly score
ω_3	0.30	Contribution of Risk Spike to the anomaly score
τ_C	0.35	Fail-closed threshold for insufficient convergence
τ_A	0.75	Abort threshold for anomaly score
τ_R	0.85	Exclusion threshold for high-risk candidates
τ_P	0.80	Threshold for repetition anomaly
τ_{root}	0.50	Maximum admissible RootGoal drift
τ_{exec}	0.70	Convergence threshold for execution permission
τ_{safe}	0.50	Medium-risk threshold that triggers confirmation
τ_{sem}	0.40	Clarification threshold for semantic ambiguity
W	3-5	Moving window size for Smooth Drift calculation

Appendix B. Minimal ReplayLog Schema

To make governance decisions auditable and reproducible, AIKernel ReplayLog should contain at least the following JSON structure. The canonical ReplayLog schema is defined in Paper 05: AIKernel Execution Model.

```
{
  "transaction_id": "tx_89a7f342-b102-4c99-b1d5-de9f939c00b2",
  "step": 4,
  "timestamp_utc": "2026-05-17T20:27:00Z",
```

```
  "kernel_version": "0.2.0",
  "policy_version": "0.2.0",
  "provider_manifest": {
    "provider_id": "string",
    "version": "string",
    "model": "string"
  },
  "goal_sovereignty": {
    "root_goal_hash": "sha256:...",
    "working_goal_hash": "sha256:...",
    "current_goal_drift": 0.1245,
    "invariant_passed": true
  },
  "observed_metrics": {
    "semantic_overlap": 0.7321,
    "goal_alignment": 0.6812,
    "smooth_drift": 0.0452,
    "normalized_entropy": 0.1892,
    "risk_score": 0.2141,
    "repetition_score": 0.0312,
    "root_goal_drift": 0.1245
  },
  "governance_scores": {
    "convergence_score": 0.6842,
    "anomaly_score": 0.0215,
    "thresholds_applied": {
      "tau_c": 0.35,
      "tau_a": 0.75,
      "tau_r": 0.85,
      "tau_root": 0.50
    }
  },
  "adjudication": {
    "decision": "Permit",
    "triggered_policy_code": "POL_OK_CONTINUOUS_CONVERGENCE",
    "selected_candidate": {
      "candidate_index": 0,
      "score": 0.8952,
      "is_tool_call": true,
      "target_capability": "Workspace.Vfs.WriteFile"
    }
  },
  "hashes": {
    "context_snapshot_hash": "sha256:...",
    "prompt_artifact_hash": "sha256:...",
    "execution_outcome_hash": "sha256:..."
  }
}
```


Appendix C. Implementation Mapping

This paper does not define implementation details as canonical. Specific C# interfaces, DTOs, provider integration mechanisms, and implementation contracts such as `ITrajectoryGovernor` are handled in Paper 07: AIKernel.NET Implementation.

However, this model maps at least to the following implementation-level abstractions:

- `TrajectoryMetrics`

- TrajectoryGovernanceResult
- TrajectoryDisposition
- ITrajectoryGovernor
- ReplayLog
- ContextSnapshot
- PolicyDecisionPoint
- PolicyEnforcementPoint